

## Memoization in Functional Programming

Not all functions can be memoized. To see why, we must discuss *functional programming*, a programming paradigm that emphasizes writing functions that don't modify global variables or any *external state* (such as files on the hard drive, internet connections, or database contents). Some programming languages, such as Erlang, Lisp, and Haskell, are heavily designed around functional programming concepts. But you can apply functional programming features to almost any programming language, including Python and JavaScript.

Functional programming includes the concepts of deterministic and nondeterministic functions, side effects, and pure functions. The `sqrt()` function mentioned in the introduction is a *deterministic* function because it always returns the same value when passed the same argument. However, Python's `random.randint()` function, which returns a random integer, is *nondeterministic* because even when passed the same arguments, it can return different values. The `time.time()` function, which returns the current time, is also nondeterministic because time is constantly moving forward.

*Side effects* are any changes a function makes to anything outside of its own code and local variables. To illustrate this, let's create a `subtract()` function that implements Python's subtraction operator (-):

### Python

---

```
>>> def subtract(number1, number2):
...     return number1 - number2
...
>>> subtract(123, 987)
-864
```

---

This `subtract()` function has no side effects; calling this function doesn't affect anything in the program outside of its code. There's no way to tell from the program's or the computer's state whether the `subtract()` function has been called once, twice, or a million times before. A function might modify local variables inside the function, but these changes are local to the function and remain isolated from the rest of the program.

Now consider an `addToTotal()` function, which adds the numeric argument to a global variable named `TOTAL`: